

Sensei Desk: Pan-Tilt AI Tutor

Sep 22, 2026 · @Someone

1. Project overview

Sensei Desk is a desk-mounted AI tutor: a phone camera on a motorized pan-tilt head that watches a student work on paper and guides them with Socratic questions, fully offline.

It is the physical version of Sensei, our multilingual Socratic tutor. Sensei already reads handwritten work, finds the exact step where a student went wrong, and asks a guiding question instead of giving the answer. Sensei Desk adds a body: a pan-tilt head we design and build, so Sensei can choose where to look (`look_at_notebook()` , `look_at_student()`) and follow the student's work across the page.

Pitch: "A personal AI tutor that sees your handwritten work, listens to your questions, and guides you step by step, while you learn on paper, with no internet required."

Context: Antler × Austin Hardtech Hackathon, week of Sept 21, 2026. Build days Tue–Thu at Assembly and HiCam. Submission deadline **Friday at noon**; live demos at the Deep Tech Week closing showcase. Judging weighs technical completeness, creativity/frontier factor, and practical potential. A working prototype is the minimum bar.

2. Goals

The Friday goal is one complete tutoring loop, running offline on hardware we built, that works every time in front of judges.

Goal	What it means	Horizon
Hackathon	Working prototype demo by Friday 12:00; credible hard-tech story	This week
Product	Desk tutor that sees paper work and guides it Socratically, offline	Next 3–6 months
Mission	Affordable one-on-one tutoring (and later hands-on science) for students without access, starting with Bangladesh	Long term

Goal	What it means	Horizon
Personal learning	Bluetooth reverse engineering (DJI gimbal), as a side project	Evenings, no demo dependency

Friday success criteria

- ☐ Pan-tilt head moves smoothly between at least 2 presets (notebook, student) on command from Python
- ☐ Camera frame of the notebook is flattened (perspective-corrected) and read by the vision model
- ☐ Sensei detects a planted mistake in a handwritten algebra problem and asks a guiding question aloud
- ☐ Student corrects the step; Sensei recognizes the fix
- ☐ Whole loop runs with the network cable unplugged
- ☐ 5 consecutive clean demo runs on Thursday night
- ☐ Backup demo video recorded

3. System architecture and domain model

Three layers: the pan-tilt head (ESP32 + servos), the camera (Pixel 2 XL), and Sensei (vision + tutor + speech) running on the local machine.

```

flowchart LR
    P[Pixel 2 XL<br/>camera stream] -->|frames over USB/Wi-Fi| V[Vision pipeline<br/>deskew + read]
    V --> T[Sensei tutor<br/>find the mistake]
    T --> S[Speech out<br/>guiding question]
    M[Microphone] --> T
    T -->|look_at_notebook| C[PanTilt client<br/>Python]
    C -->|serial: PRESET notebook| E[ESP32 firmware]
    E --> SV[Pan + tilt servos]
    SV -.moves.-> P

```

The tutor decides where to look; the client turns that into serial commands; the ESP32 moves the servos smoothly; the camera sees the new view.

Domain model

Entity	Meaning	Key fields
Gaze	Where the camera points	pan_deg , tilt_deg , preset
Preset	Named, calibrated gaze	name (notebook, student, home), angles
Frame	One camera image, deskewed	timestamp , gaze , page_quad
WorkStep	One line of the student's handwritten solution	index , latex , confidence
Mistake	The first incorrect step	step_index , kind (sign, arithmetic, rule)
Hint	Socratic question tied to a mistake	text , level (nudge → stronger)
Session	One tutoring interaction	problem , steps[] , hints_given[] , resolved

The tutor loop reasons over `Session`: read the new `WorkStep` s, find the first `Mistake` , pick the next `Hint` level, and check whether the next frame resolves it. The hardware only ever sees `Gaze` and `Preset` .

4. Equipment and bill of materials

Budget about \$65–90 if the maker space supplies the small parts, about \$115–160 if we buy everything. Prices are approximate Amazon ranges, not checked today. Filter by "Get it tomorrow" so parts land Wednesday; ask Assembly/HiCam what they already stock before ordering.

Item	Qty	Purpose	Approx. price	Must have?
Pan-tilt kit with 2× 996R servos (or pre-assembled equivalent)	1	Brackets, bearing, servos	\$20–30	Yes
MG996R servo 4-pack	1	Spares for a burned-out servo	\$18–25	Strongly advised

Item	Qty	Purpose	Approx. price	Must have?
ESP32-WROOM-32 dev board (USB-C, 2–3 pack)	1 pack	Servo controller, serial link to laptop	\$15–20	Yes
5V $\geq$ 4A power supply, 5.5×2.1 mm barrel	1	Servo power (never from ESP32/USB)	\$10–14	Yes
DC barrel jack → screw terminal adapter	1 pack	Connect supply to servo rails	\$6–8	Yes
Phone tripod clamp, 1/4"-20 thread	1	Holds the Pixel on the tilt bracket	\$8–12	Yes
1/4"-20 bolts/nuts + M3 screw assortment	1	Clamp-to-bracket, bracket-to-base	\$10–15	Yes
Jumper wires (M-F, M-M)	1 pack	Signal and power wiring	\$6–8	Maker space?
Electrolytic capacitors, 1000 μ F $\geq$ 10 V	1 assortment	Smooth servo current spikes	\$8–10	Maker space?
Half-size breadboard	1	Power rails, common ground	\$6–9	Maker space?
Desk C-clamp	1	Anchor base to table	\$8–10	Optional

Already have: Pixel 2 XL (camera), laptop / local GPU box running Sensei, USB cable for the ESP32.

From the maker space: scrap wood or plywood block for a heavy base (or 3D-print a weighted base), drill, screwdriver set, multimeter, soldering iron (optional).

5. Build part 1: mechanical assembly

Goal: a rigid, balanced head that holds the Pixel steady at any preset angle. Allow 2–3 hours.

1. **Center both servos before assembly.** Power each servo briefly and command 90° (use the firmware in section 7, or any servo tester). Assembling off-center is the most common reason a head can't reach one of its presets.

2. **Mount the pan servo to the base.** Screw it to a heavy wood block (at least about 15 × 10 × 2 cm) or a 3D-printed weighted base. The base must not tip when the phone tilts forward. C-clamp it to the table for the demo.
3. **Attach the U-bracket to the pan servo horn** using the kit's bearing, so the bearing carries the weight instead of the servo shaft.
4. **Mount the tilt servo between the U-bracket uprights**, with its shaft on the tilt axis.
5. **Bolt the phone clamp to the tilt bracket** with a 1/4"-20 bolt. Mount it as low and as close to the tilt axis as possible.
6. **Balance check.** Insert the Pixel. With servos unpowered, the head should not swing hard toward one side. If it does, move the clamp closer to the axis or add a small counterweight (a few washers) on the opposite side.
7. **Measure the camera geometry.** With the phone at the notebook preset, note the camera height above the desk and the angle to the page center. You will use these for perspective correction.

Tips

- Route servo cables with slack through the pan rotation. A tight cable acts like a spring and fights the servo.
- Mark a fixed spot for the notebook on the desk with tape so the preset always frames the page.
- Point the camera away from bright windows; add a desk lamp so the page is evenly lit.

6. Build part 2: wiring and power

Servos run from the 5V supply; the ESP32 runs from laptop USB; both grounds are tied together. Allow 30–45 minutes.

flowchart LR

```

PSU[5V 5A supply] --> R[Breadboard rails<br/>+5V / GND]
CAP[1000 µF cap] --- R
R -->|red +5V| PS[Pan servo]
R -->|red +5V| TS[Tilt servo]
R -->|GND| E[ESP32 GND]
E -->|GPIO 18 signal| PS
E -->|GPIO 19 signal| TS
L[Laptop USB] -->|power + serial| E

```

From	To	Wire
Supply +5V	Breadboard + rail	Screw terminal adapter
Supply GND	Breadboard – rail	Screw terminal adapter
Capacitor + / –	+ rail / – rail	Mind polarity: stripe = –
Pan servo red / brown	+ rail / – rail	Servo lead
Tilt servo red / brown	+ rail / – rail	Servo lead
Pan servo orange (signal)	ESP32 GPIO 18	Jumper
Tilt servo orange (signal)	ESP32 GPIO 19	Jumper
ESP32 GND	Breadboard – rail	Jumper (shared ground)
ESP32 USB	Laptop	USB cable

Rules that prevent most failures

- Never power the servos from the ESP32's 5V/VIN pin or from the laptop. A loaded MG996R pulls well over 1 A when starting and will reset the ESP32.
- Always connect the ESP32 GND to the servo supply GND. Without a shared ground, the servos jitter or ignore commands.
- Check polarity with the multimeter before plugging in the servos. Reversed 5V can destroy a servo instantly.
- The ESP32's 3.3 V signal usually drives MG996R servos fine. If a servo is erratic, test with a shorter signal wire before suspecting the firmware.

7. Build part 3: ESP32 firmware

The firmware accepts one-line text commands over USB serial (115200 baud) and moves each servo toward its target one degree at a time, so motion is smooth. Allow 1 hour including setup.

Setup

1. Install the Arduino IDE 2.x (or PlatformIO in VS Code).
2. Add the ESP32 board package (Espressif "esp32") via Boards Manager.
3. Install the **ESP32Servo** library via Library Manager.

4. Select your board (e.g. "ESP32 Dev Module") and the USB port, then upload the sketch below.

Command protocol

Command	Example	Reply
PING	PING	PONG
PAN <deg>	PAN 60	OK ,later ARRIVED 60 90
TILT <deg>	TILT 135	OK ,later ARRIVED ...
MOVE <pan> <tilt>	MOVE 90 135	OK ,later ARRIVED ...
PRESET <name>	PRESET notebook	OK or ERR unknown preset
SPEED <ms/step>	SPEED 10	OK (lower = faster)
POS?	POS?	POS 90 135

Angles are servo degrees, 0-180, 90 = centered. Limits clamp every command so the head can never hit its own bracket.

```
#include <Arduino.h>
#include <ESP32Servo.h>

const int PAN_PIN = 18;
const int TILT_PIN = 19;

// Safe mechanical limits in degrees. Calibrate on your build (section 9).
const int PAN_MIN = 10, PAN_MAX = 170;
const int TILT_MIN = 40, TILT_MAX = 150;

struct Preset { const char* name; int pan; int tilt; };
// Placeholder angles: replace with your calibrated values.
Preset presets[] = {
    {"home",    90,  90},
    {"notebook", 90, 135},
    {"student",  90,  70},
};
const int NUM_PRESETS = sizeof(presets) / sizeof(presets[0]);

Servo panServo, tiltServo;
```

```

int panPos = 90, tiltPos = 90;
int panTarget = 90, tiltTarget = 90;
int stepMs = 15;           // ms per 1-degree step (~66 deg/s)
bool moving = false;
unsigned long lastStep = 0;
String buf;

int clampi(int v, int lo, int hi) { return v < lo ? lo : (v > hi ? hi : v); }

void setTargets(int p, int t) {
    panTarget = clampi(p, PAN_MIN, PAN_MAX);
    tiltTarget = clampi(t, TILT_MIN, TILT_MAX);
    moving = true;
}

void handleLine(String line) {
    line.trim();
    if (line.length() == 0) return;
    int sp = line.indexOf(' ');
    String cmd = (sp < 0) ? line : line.substring(0, sp);
    String arg = (sp < 0) ? "" : line.substring(sp + 1);
    cmd.toUpperCase();
    arg.trim();

    if (cmd == "PING") {
        Serial.println("PONG");
    } else if (cmd == "PAN") {
        setTargets(arg.toInt(), tiltTarget); Serial.println("OK");
    } else if (cmd == "TILT") {
        setTargets(panTarget, arg.toInt()); Serial.println("OK");
    } else if (cmd == "MOVE") {
        int sp2 = arg.indexOf(' ');
        if (sp2 < 0) { Serial.println("ERR usage: MOVE <pan> <tilt>"); return; }
        setTargets(arg.substring(0, sp2).toInt(), arg.substring(sp2 +
1).toInt());
        Serial.println("OK");
    } else if (cmd == "PRESET") {
        arg.toLowerCase();
        for (int i = 0; i < NUM_PRESETS; i++) {
            if (arg == presets[i].name) {
                setTargets(presets[i].pan, presets[i].tilt);
                Serial.println("OK");
                return;
            }
        }
    }
}

```

```

    }
  }
  Serial.println("ERR unknown preset");
} else if (cmd == "SPEED") {
  stepMs = clampi(arg.toInt(), 2, 100); Serial.println("OK");
} else if (cmd == "POS?") {
  Serial.printf("POS %d %d\n", panPos, tiltPos);
} else {
  Serial.println("ERR unknown command");
}
}

void setup() {
  Serial.begin(115200);
  ESP32PWM::allocateTimer(0);
  ESP32PWM::allocateTimer(1);
  panServo.setPeriodHertz(50);
  tiltServo.setPeriodHertz(50);
  panServo.attach(PAN_PIN, 500, 2400); // pulse range in microseconds
  tiltServo.attach(TILT_PIN, 500, 2400);
  panServo.write(panPos);
  tiltServo.write(tiltPos);
  Serial.println("READY");
}

void loop() {
  // 1. Read serial commands, one per line.
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n') { handleLine(buf); buf = ""; }
    else if (c != '\r' && buf.length() < 64) { buf += c; }
  }

  // 2. Step servos toward targets for smooth motion.
  if (moving && millis() - lastStep >= (unsigned long)stepMs) {
    lastStep = millis();
    if (panPos != panTarget) panPos += (panTarget > panPos) ? 1 : -1;
    if (tiltPos != tiltTarget) tiltPos += (tiltTarget > tiltPos) ? 1 : -1;
    panServo.write(panPos);
    tiltServo.write(tiltPos);
    if (panPos == panTarget && tiltPos == tiltTarget) {
      moving = false;
    }
  }
}

```

```

        Serial.printf("ARRIVED %d %d\n", panPos, tiltPos);
    }
}
}

```

If a servo moves the wrong way for a preset, don't rewire: swap the preset angle around 90 (e.g. 135 ↔ 45). If full travel is short, widen the pulse range (e.g. 500–2500 µs) a little at a time and watch for buzzing at the ends.

8. Build part 4: Python client and Sensei integration

A small `PanTilt` class hides the serial protocol, so Sensei's tutor code only calls `look_at_notebook()` or `look_at_student()`. Allow 2–3 hours including camera setup.

Install: `pip install pyserial opencv-python numpy`

8.1 PanTilt client

```

import threading
import time

import serial  # pyserial

class PanTilt:
    """Talks to the Sensei Desk ESP32 firmware over USB serial."""

    def __init__(self, port: str, baud: int = 115200, settle_s: float = 0.3):
        self.ser = serial.Serial(port, baud, timeout=0.2)
        self.settle_s = settle_s          # let the phone stop wobbling
        self._lock = threading.Lock()
        time.sleep(2.0)                   # ESP32 reboots when the port opens
        self.ser.reset_input_buffer()
        if self._send("PING") != "PONG":
            raise RuntimeError("Pan-tilt head not responding")

    # --- low level -----
    def _readline(self, deadline: float) -> str:
        while time.monotonic() < deadline:
            line = self.ser.readline().decode(errors="ignore").strip()
            if line:
                return line

```

```

        raise TimeoutError("No reply from pan-tilt head")

    def _send(self, text: str, wait_arrival: bool = False, timeout: float =
6.0):
        with self._lock:
            self.ser.reset_input_buffer()      # drop stale ARRIVED lines
            self.ser.write((text + "\n").encode())
            deadline = time.monotonic() + timeout
            reply = self._readline(deadline)
            if reply.startswith("ERR"):
                raise ValueError(f"{text!r} -> {reply}")
            if not wait_arrival:
                return reply
            while True:
                line = self._readline(deadline)
                if line.startswith("ARRIVED"):
                    time.sleep(self.settle_s)
                    _, pan, tilt = line.split()
                    return int(pan), int(tilt)

# --- public API -----
def move(self, pan: int, tilt: int):
    return self._send(f"MOVE {pan} {tilt}", wait_arrival=True)

def preset(self, name: str):
    return self._send(f"PRESET {name}", wait_arrival=True)

def speed(self, ms_per_step: int):
    self._send(f"SPEED {ms_per_step}")

def position(self):
    _, pan, tilt = self._send("POS?").split()
    return int(pan), int(tilt)

def look_at_notebook(self):
    return self.preset("notebook")

def look_at_student(self):
    return self.preset("student")

def home(self):
    return self.preset("home")

```

```

def close(self):
    self.ser.close()

if __name__ == "__main__":
    head = PanTilt("/dev/ttyUSB0")    # Windows: "COM3"; macOS:
    "/dev/cu.usbserial-..."
    for _ in range(3):
        print("notebook", head.look_at_notebook())
        print("student", head.look_at_student())
    head.home()

```

8.2 Camera from the Pixel 2 XL

The Pixel 2 XL tops out at Android 11, so it has no built-in USB webcam mode. Use a webcam app instead:

- **IP Webcam** (or DroidCam) streams MJPEG over the local network:
`cv2.VideoCapture("http://<phone-ip>:8080/video")`.
- For the offline demo, put the phone and laptop on a local hotspot or USB tethering. No internet is needed, only a local link.
- Lock focus and exposure in the app so the image doesn't pump when the head moves.

8.3 Page capture and perspective correction

The notebook sits at a taped, fixed spot, so the most reliable method is a **calibrated page quad**: click the four page corners once, save them, and reuse them for every frame. Auto-detection is a fallback for when the page moves.

```

import json
import time

import cv2
import numpy as np

OUT_W, OUT_H = 1240, 1754    # A4 aspect at ~150 dpi

def order_corners(pts):
    pts = np.asarray(pts, dtype="float32")
    s, d = pts.sum(axis=1), np.diff(pts, axis=1).ravel()
    return np.array([pts[s.argmax()], pts[d.argmax()]],    # top-left, top-

```

```

right
            pts[s.argmax()], pts[d.argmax()]], # bottom-right,
bottom-left
            dtype="float32")

def deskew(frame, quad):
    dst = np.array([[0, 0], [OUT_W, 0], [OUT_W, OUT_H], [0, OUT_H]],
dtype="float32")
    m = cv2.getPerspectiveTransform(order_corners(quad), dst)
    return cv2.warpPerspective(frame, m, (OUT_W, OUT_H))

def auto_find_page(frame):
    gray = cv2.GaussianBlur(cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY), (5, 5),
0)
    edges = cv2.Canny(gray, 50, 150)
    contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    min_area = 0.2 * frame.shape[0] * frame.shape[1]
    for c in sorted(contours, key=cv2.contourArea, reverse=True)[:5]:
        approx = cv2.approxPolyDP(c, 0.02 * cv2.arcLength(c, True), True)
        if len(approx) == 4 and cv2.contourArea(approx) > min_area:
            return approx.reshape(4, 2)
    return None

def capture_page(head, cam, quad_path="page_quad.json"):
    head.look_at_notebook()
    for _ in range(5):
        cam.grab() # flush frames buffered during the
move
    ok, frame = cam.read()
    if not ok:
        raise RuntimeError("Camera read failed")
    try:
        quad = json.load(open(quad_path))
    except FileNotFoundError:
        quad = auto_find_page(frame)
    return deskew(frame, quad) if quad is not None else frame

```

8.4 Tutor loop

Wire the hardware into Sensei's existing vision and tutoring code. The `sensei.*` calls below are placeholders for your current functions.

```
head = PanTilt("/dev/ttyUSB0")
cam = cv2.VideoCapture("http://192.168.43.1:8080/video")
session = sensei.new_session(problem="Solve 3x + 5 = 20")

while not session.resolved:
    page = capture_page(head, cam)
    steps = sensei.read_steps(page)          # WorkStep[]
    mistake = sensei.first_mistake(session, steps)
    if mistake is None:
        if sensei.is_complete(session, steps):
            head.look_at_student()
            sensei.say("Nice work. Can you explain why that last step
works?")
            session.resolved = True
        else:
            hint = sensei.next_hint(session, mistake) # escalates nudge ->
stronger
            head.look_at_student()                # face the student while
speaking
            sensei.say(hint.text)
            time.sleep(2.0)                       # let the student write
```

9. Testing plan

Test bottom-up: each layer passes before the next is added, so a failure always points to one layer.

9.1 Bench tests (hardware)

#	Test	How	Pass when
H1	Supply polarity	Multimeter on rails before servos go in	+5.0 V (± 0.25) on the + rail
H2	Serial link	Serial monitor at 115200, send PING	READY on boot, PONG reply

#	Test	How	Pass when
H3	Single servo	PAN 45 , PAN 135 , PAN 90 with no phone	Smooth motion, ARRIVED each time
H4	Limits	PAN 0 , TILT 180	Clamped to limits; no buzzing or bracket contact
H5	Loaded tilt	Phone mounted, PRESET notebook , hold 60 s	No sag, no hum, ESP32 doesn't reset
H6	Brownout	Rapid MOVE 20 40 / MOVE 160 150 × 10	ESP32 never prints READY again mid-test
H7	Repeatability	PRESET notebook 20× from random positions; photo each time	Page corners land within about 1 cm
H8	Endurance	Script alternates presets for 15 min	Servos warm, not hot; no drift

9.2 Calibration

1. Jog with MOVE until the page fills the frame; record POS? as the notebook preset.
2. Repeat for the student's face and a neutral home position.
3. Paste the values into presets[] and re-flash.
4. Grab one frame at the notebook preset, click the 4 page corners, and save them to page_quad.json .

9.3 Software and integration tests

#	Test	Pass when
S1	python pantilt.py demo loop	3 round trips, no timeout
S2	Camera stream	≥10 fps at 720p over the local link
S3	Deskew	Printed page looks flat and rectangular; lines are straight
S4	Read steps	5 handwritten solutions: every line read correctly

#	Test	Pass when
S5	Find mistake	5 planted mistakes (sign, arithmetic, rule): first wrong step found each time
S6	Hint quality	Hints ask a question and never state the answer
S7	Fix detection	After the correction, Sensei moves on instead of repeating the hint
S8	Offline	Wi-Fi internet off, cable out: full loop still works
S9	End-to-end latency	Frame captured → hint spoken in under about 8 s

9.4 Demo rehearsal checklist

- ☐ 5 clean full runs in a row on the actual demo table
- ☐ Tested under venue-like lighting (bright, overhead) and with a noisy room
- ☐ Spare servo, spare ESP32 and a flashed backup firmware on hand
- ☐ Phone charged; laptop on power; USB cables taped down
- ☐ Backup video of a perfect run on the laptop desktop
- ☐ One teammate owns "reset the demo" (new page, home preset, new session)

10. Schedule (working backward from Friday noon)

Features freeze Thursday evening; Friday morning is only for rehearsal and submission.

When	Hardware track	Software track	Exit criterion
Fri, before 12:00	Final check, pack spares	Submit, final run-through	Submitted; demo runs clean
Thu evening	Feature freeze	Record backup video, polish pitch	5 clean runs in a row (9.4)
Thu day	Endurance and lighting tests (H8)	Hint quality, fix detection, offline (S6–S9)	Full loop offline
Wed evening	Calibrate presets and page quad (9.2)	<code>capture_page</code> + tutor loop (8.3–8.4)	First end-to-end hint spoken

When	Hardware track	Software track	Exit criterion
Wed day	Mechanical build + wiring (sections 5–6), H1–H7	Client + camera stream (S1–S3)	Head moves between presets on command
Tue (today)	Order parts; flash firmware on bare ESP32; build base	Write <code>PanTilt</code> client against the serial monitor; set up IP Webcam	Parts arriving Wed; firmware replies <code>PONG</code>

Evenings (optional, not on the critical path): DJI gimbal Bluetooth capture and analysis (section 12).

11. Demo script and pitch

The demo is about 3 minutes: one problem, one planted mistake, one guided fix, then the cable pull.

Time	On stage	What judges see
0:00	Pitch line and the problem: one-on-one tutoring is the most effective way to learn, and most students never get it	Device idle at home preset
0:30	Student starts "Solve $3x + 5 = 20$ " on paper	Head turns to the notebook
1:00	Student writes $3x = 25$ (sign mistake)	Sensei: "When you moved the 5 across, what happened to its sign?" Head turns to face the student while speaking
1:40	Student fixes it: $3x = 15$, $x = 5$	Head looks back at the notebook; Sensei confirms and asks the student to explain why
2:10	Pull the network cable, repeat a short exchange	Still works: everything runs locally
2:30	Show the hardware: custom pan-tilt head, firmware, smooth presets	Close-up of the mechanism
2:50	Close: vision and next steps	

Pitch (about 30 seconds)

A personal AI tutor that sees your handwritten work, listens to your questions, and guides you step by step, while you learn on paper. Sensei Desk runs entirely offline on local hardware, so it works in classrooms and learning centers with unreliable internet. We built the pan-tilt head, firmware and vision pipeline this week. Next, we add instrumented lab stations so Sensei can guide hands-on science, starting with circuits.

Answering "Is this really hard tech?"

We designed and built the actuated head, its control firmware and the camera-to-tutor loop this week, and the tutoring depends on it: Sensei decides where to look. Our next module adds electrical sensing, so the tutor reasons from physical measurements, not only images.

12. Risks, fallbacks and stretch goals

Every risk on the critical path has a fallback ready before Thursday.

Risk	Likelihood	Fallback
Parts arrive late	Medium	Maker-space servos and ESP32; order from 2 sellers
Tilt servo can't hold the phone	Medium	Move clamp closer to the axis, add counterweight, or use the spare MG996R
ESP32 resets when servos move	Medium	Check separate supply + shared ground; add a second capacitor
Handwriting misread under venue lights	Medium	Desk lamp, locked exposure, dark pen, high-contrast paper
Phone overheats or disconnects	Low	Keep it plugged in; restart script; backup video
Live demo fails on stage	Low	Reset procedure (9.4), then backup video

Stretch goal A: circuit lab station

This extends Sensei from paper to hands-on science and makes the strongest hard-tech case. A student builds a simple LED circuit and predicts the result; an ESP32 with an INA219 current sensor and ADC channels measures the real voltage and current; Sensei compares the prediction, the camera view of the wiring, and the measurements, then asks a guiding question. Build it only if the core demo passes 9.4 by Wednesday night; otherwise it goes in the pitch as the next module.

Stretch goal B: DJI gimbal Bluetooth reverse engineering (learning project)

This is a learning project, not a demo dependency.

1. Enable Developer options on a phone running DJI Mimo, turn on "Bluetooth HCI snoop log", then toggle Bluetooth off and on.
2. Connect the gimbal, do one action at a time (pan left, 5 s pause, pan right, tilt), and note the times.
3. Pull the log with `adb bugreport`; open `btsnoop_hci.log` in Wireshark.
4. Filter ATT writes: `btatt.opcode == 0x52 || btatt.opcode == 0x12`, and match payloads to actions.
5. Try the Wireshark dissector from [xaionaro/reverse-engineering-dji](https://github.com/xaionaro/reverse-engineering-dji). DJI's other handhelds use the DUMML protocol, and [lib-osmo-ble](https://github.com/lib-osmo-ble) documents DUMML gimbal commands for the Osmo Pocket 3. Whether the Osmo Mobile uses the same protocol is unconfirmed.
6. Replay a candidate command from Python with `bleak` and see if the gimbal moves.

For a cheap gimbal whose protocol is already documented, see [OpenGimbal](https://github.com/Hohem/Honji) (Hohem/Honji family; tilt support not yet confirmed on every model).

Upgrade path after the hackathon

Replace hobby servos with brushless gimbal motors and the open-source STorM32 controller for quieter, smoother motion; design a custom enclosure; move vision and the tutor onto a single local compute box shared by a classroom.